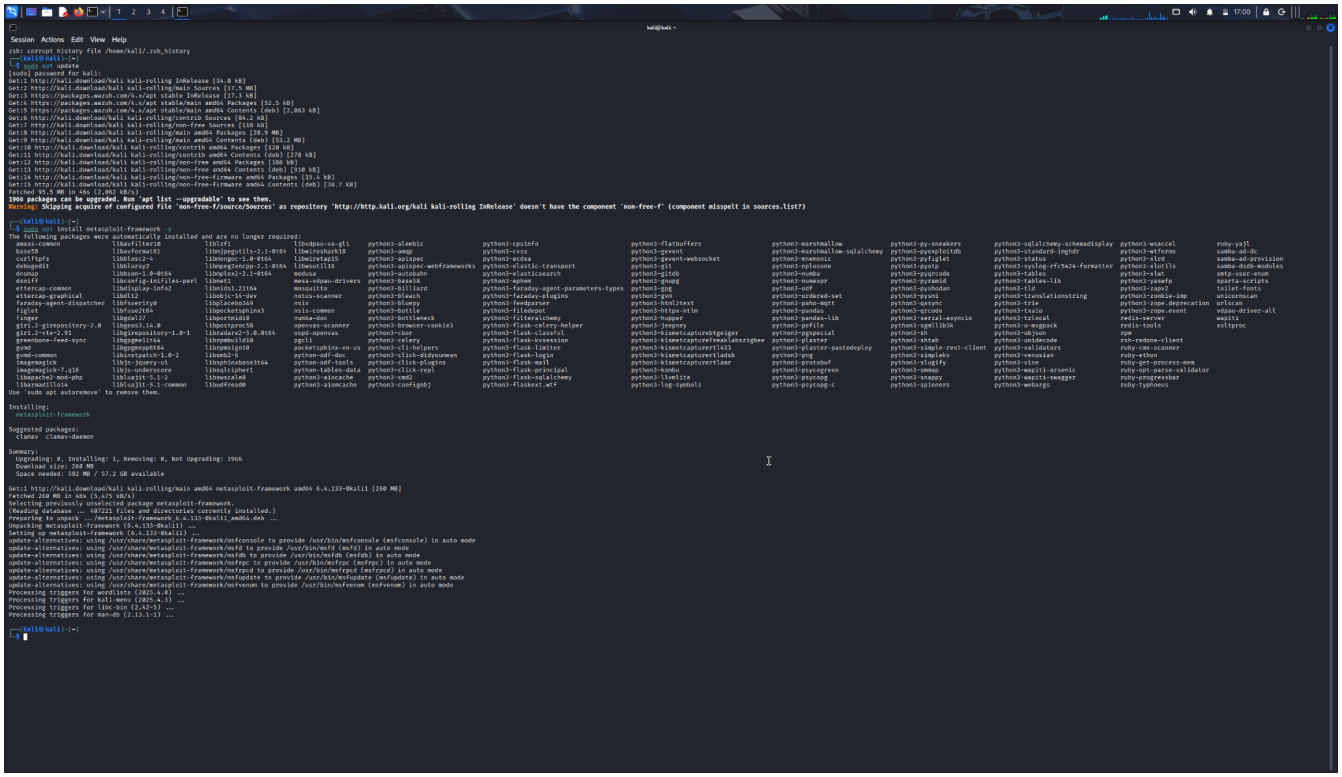
Detailed Penetration Testing Project Report

This report documents a controlled penetration testing exercise conducted within a virtual lab environment using Kali Linux, Metasploit Framework, and Windows/Metasploitable systems. The purpose of the assessment was to demonstrate the different phases of penetration testing including reconnaissance, enumeration, exploitation, payload deployment, session establishment, and post-exploitation activities.

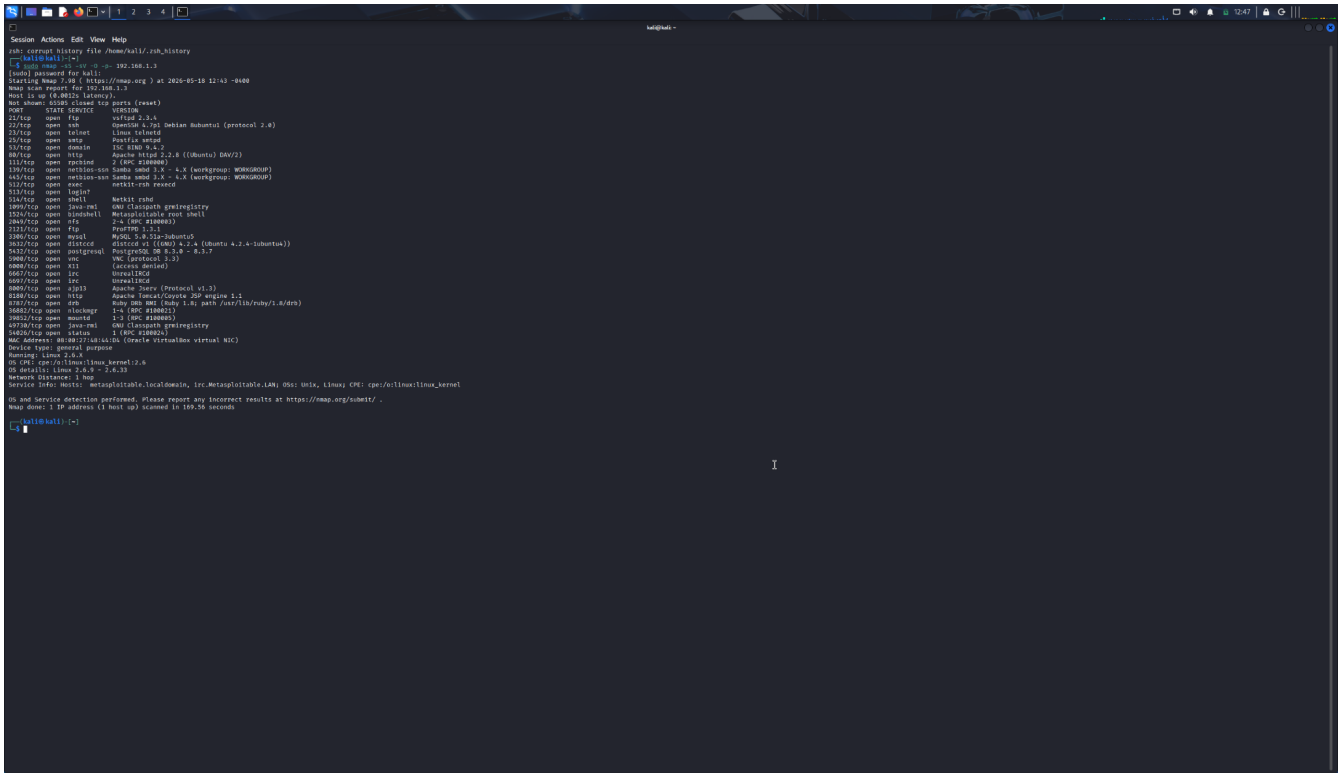
1. Lab Setup and Environment

The penetration testing lab environment consisted of multiple virtual machines connected within the same network. Kali Linux was configured as the attacker machine while Windows and Metasploitable systems served as vulnerable targets. The systems communicated through private IP addressing within the 192.168.1.0/24 subnet.



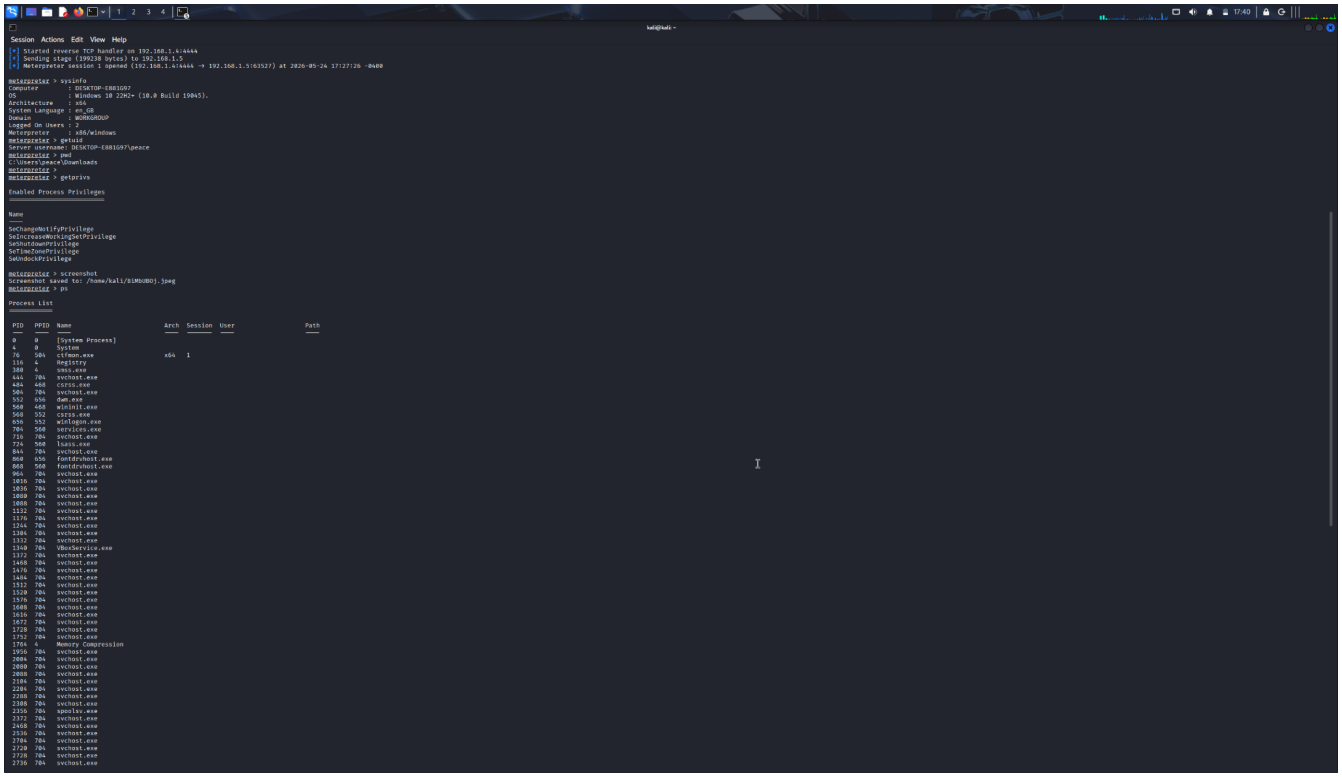
2. Reconnaissance and Port Scanning

Nmap scanning was performed against the target host using service and operating system detection techniques. The scan identified multiple open ports including FTP (21), SSH (22), HTTP (80), SMB (139/445), and MySQL (3306). Several outdated services were discovered, increasing the attack surface of the target machine.



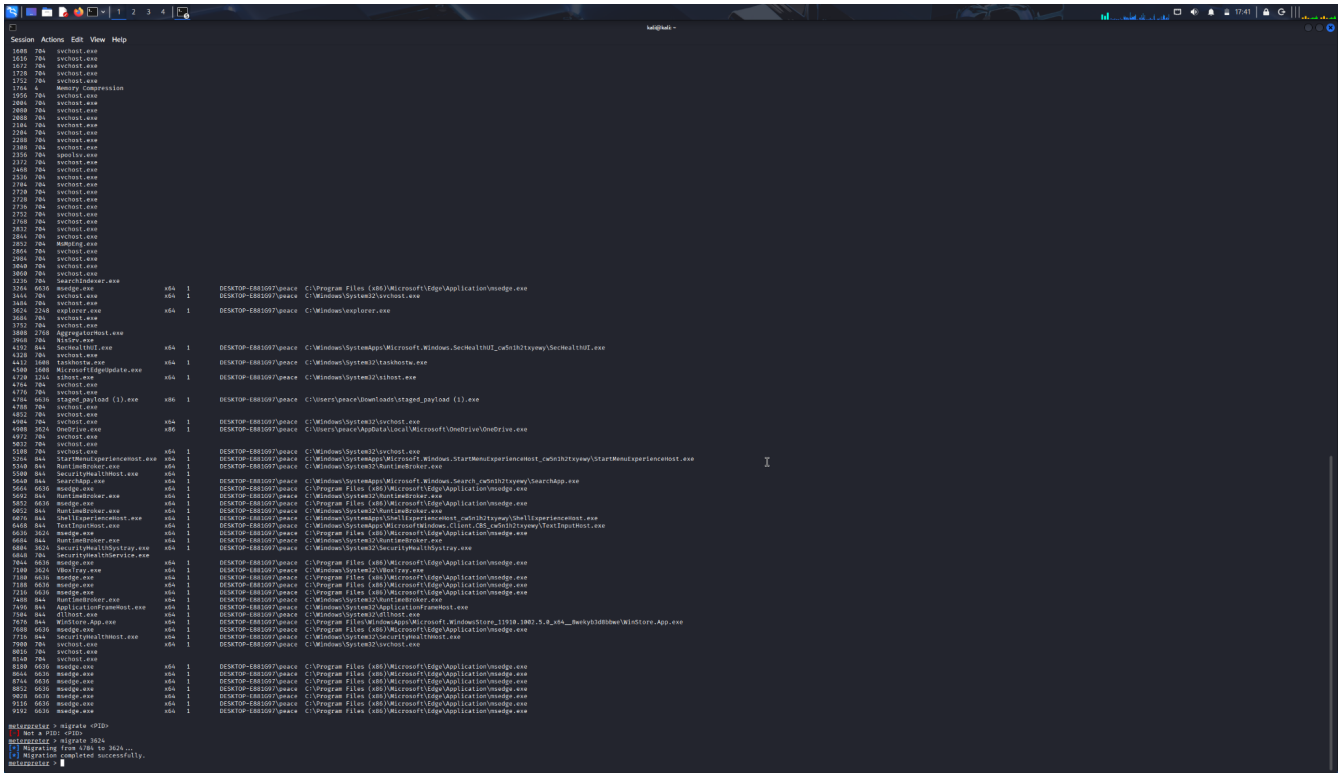
3. Enumeration Phase

Enumeration activities were conducted to gather detailed information about services running on the target machine. The identified versions of the applications and exposed services provided useful intelligence for selecting appropriate exploits within Metasploit Framework.



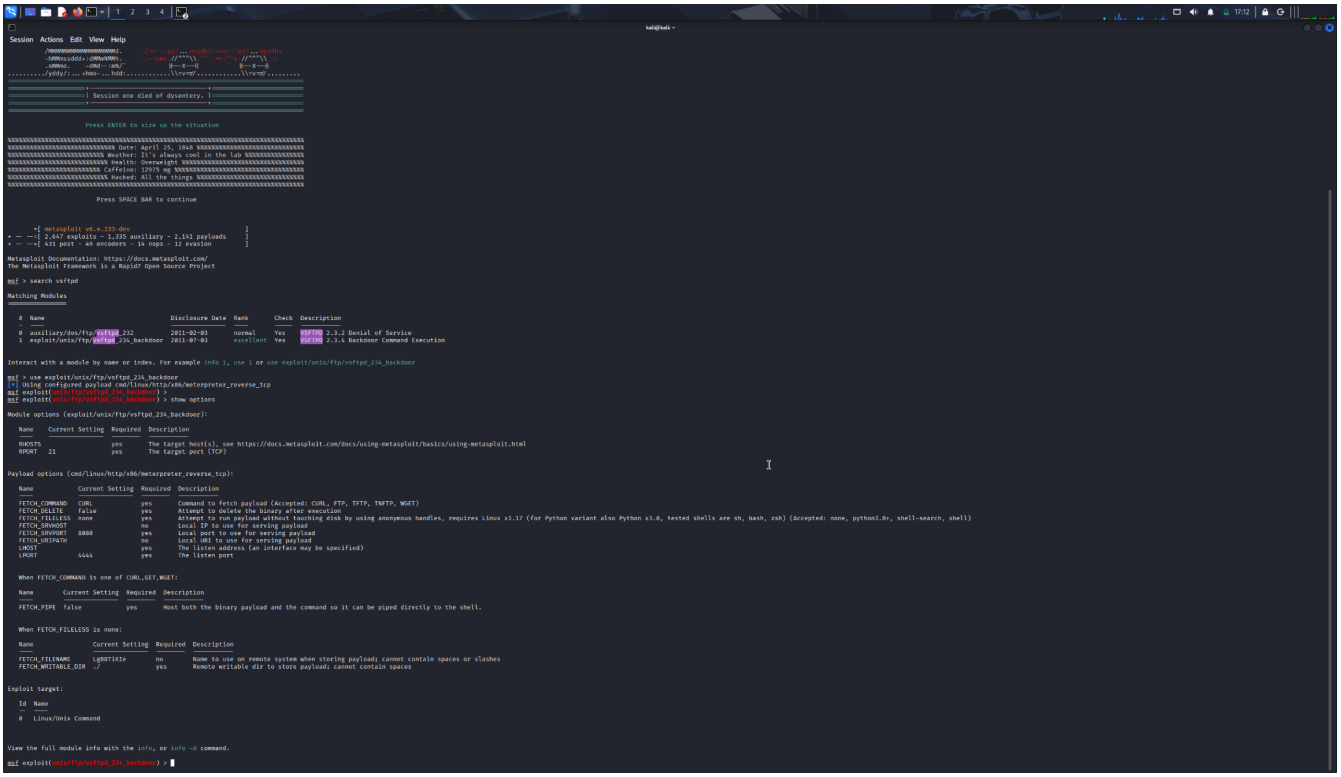
4. Additional Enumeration Results

Further enumeration revealed vulnerable services and exposed configurations on the target system. The information gathered during this phase assisted in confirming exploitable services and validating attack vectors.



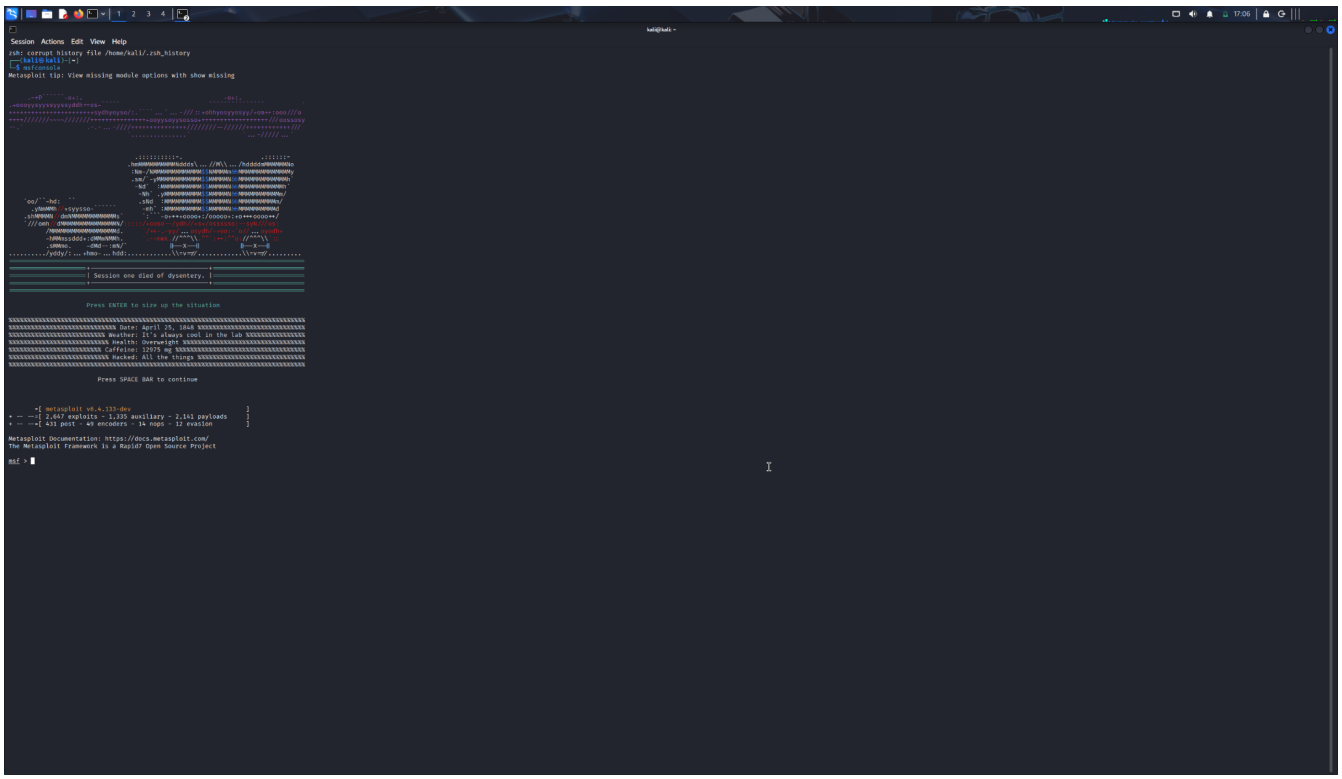
5. Exploit Identification

The Metasploit Framework search functionality was used to identify available exploit modules associated with the discovered services. The `vsftpd_234_backdoor` exploit module was identified as a potential attack vector because the target machine was running the vulnerable VSFTPD 2.3.4 service.



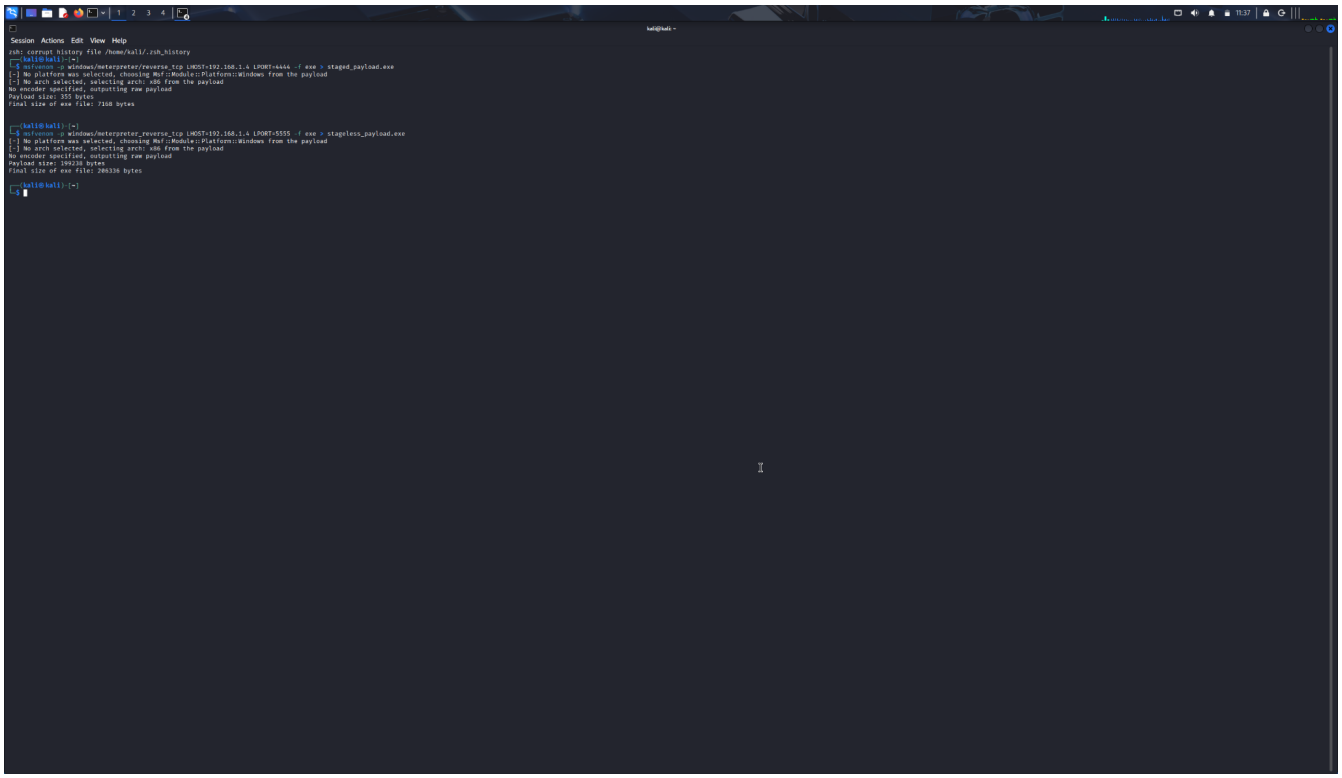
6. Metasploit Framework Configuration

Metasploit Framework was initialized and configured to handle reverse connections from the target machine. Appropriate payloads, listening hosts, and ports were configured before beginning the exploitation phase.



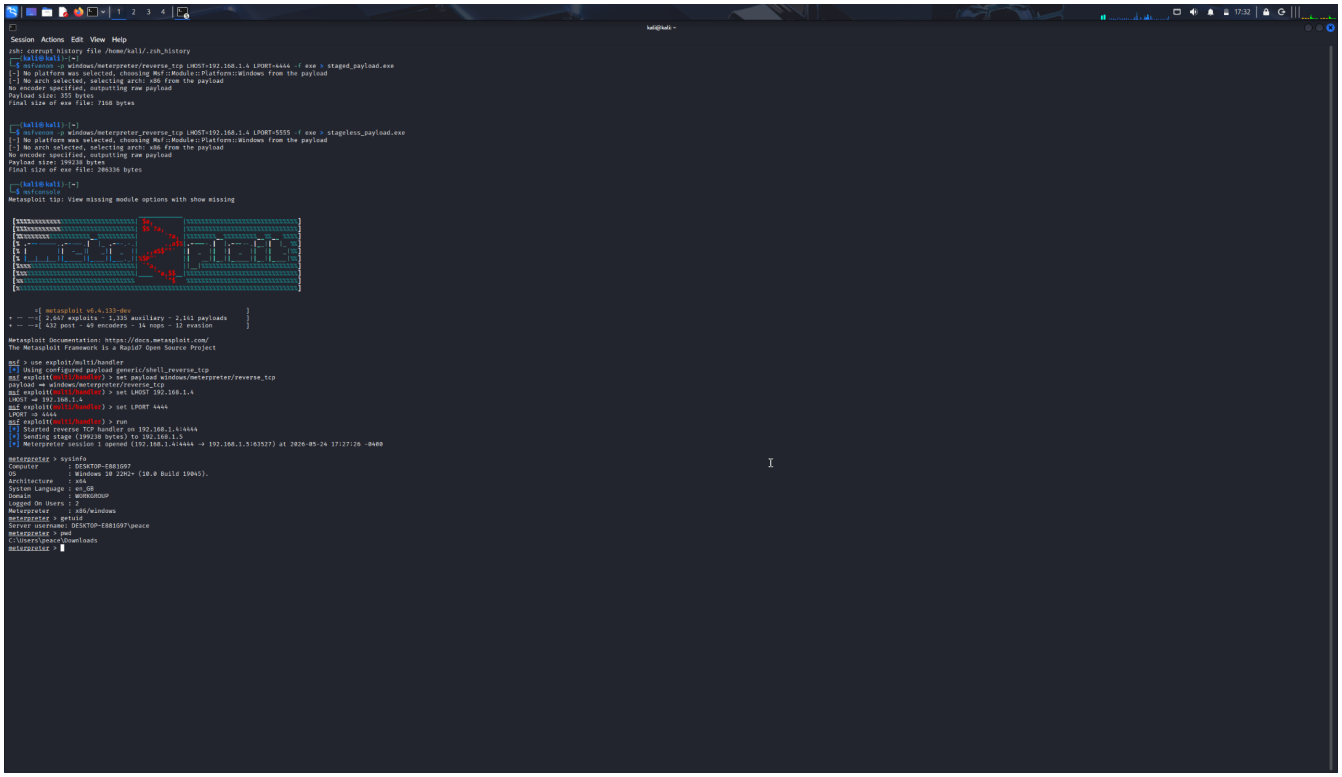
7. Payload Creation

The `msfvenom` utility was used to generate staged and stageless Meterpreter reverse TCP payloads. The payloads were configured to establish a reverse shell connection from the target system back to the attacker machine using the attacker's IP address and custom listening ports.



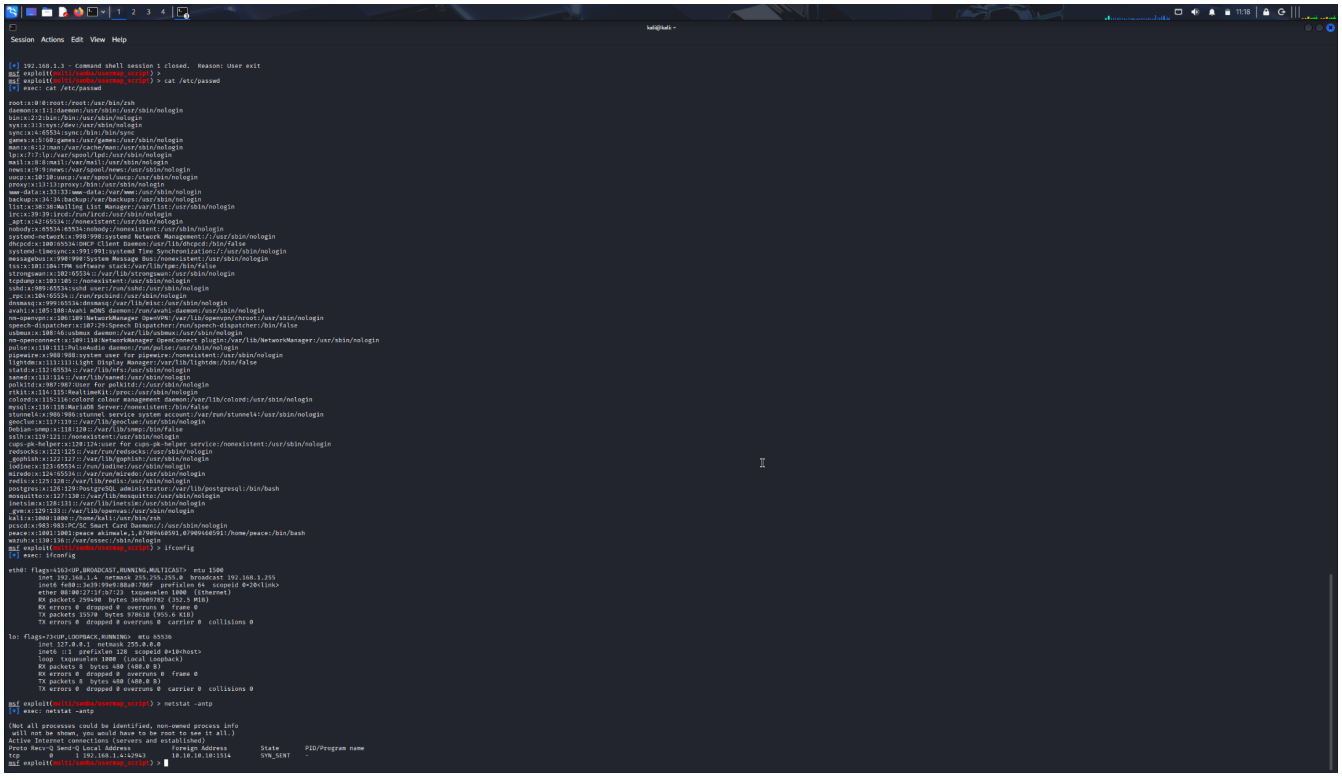
8. Successful Meterpreter Session

After executing the payload on the Windows target machine, a successful Meterpreter reverse shell session was established. The attacker machine received a reverse connection from the victim host, allowing remote command execution and system interaction.



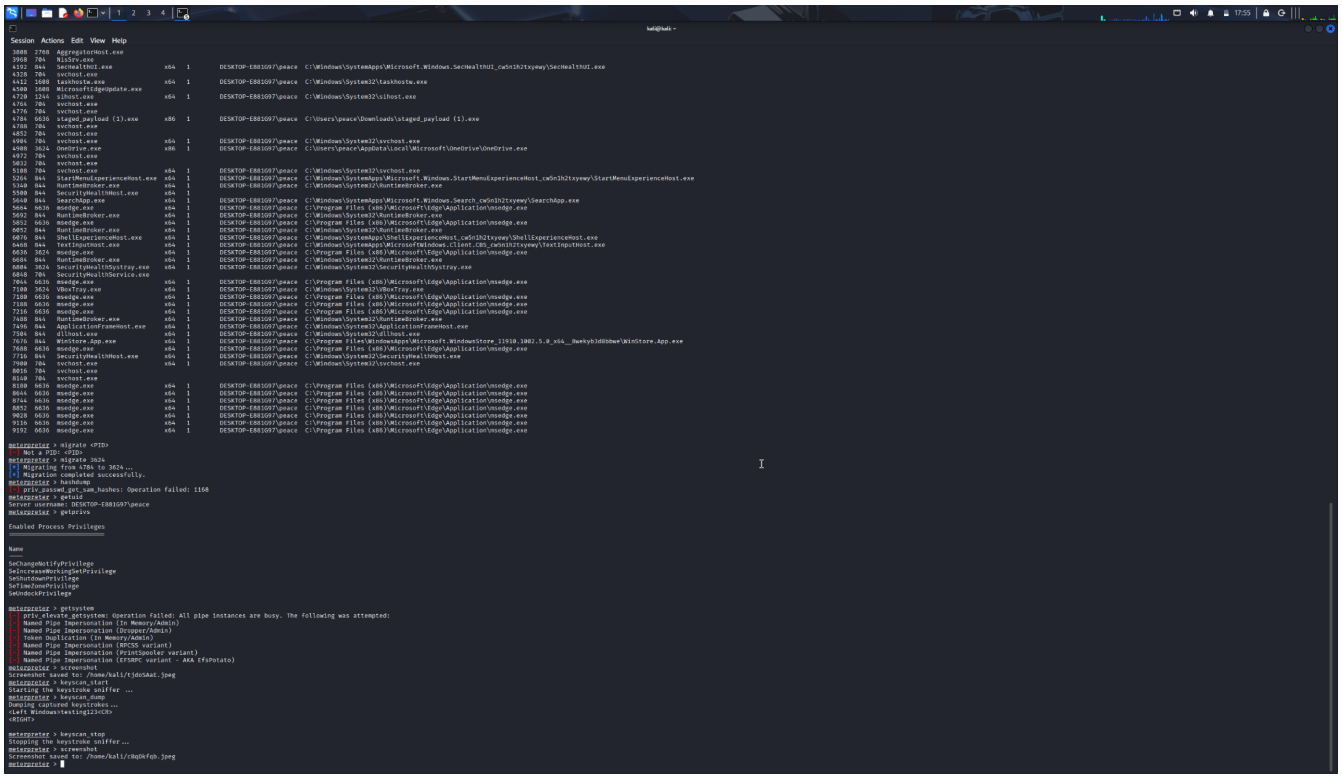
9. Post-Exploitation Activities

Post-exploitation activities included process enumeration, process migration, privilege inspection, and information gathering. Process migration was performed to improve session stability by moving the Meterpreter session into a legitimate Windows process.



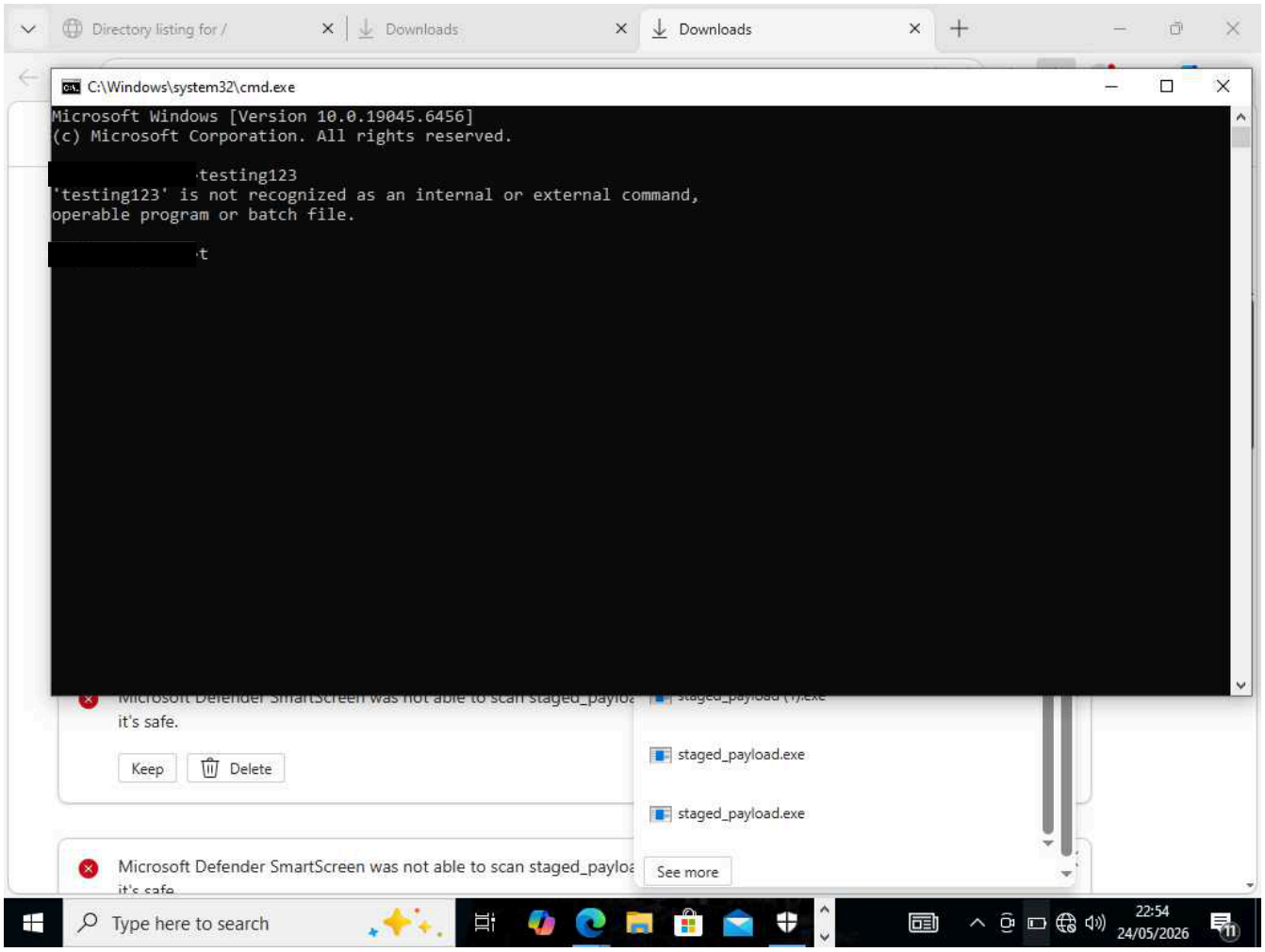
10. Keystroke Logging Demonstration

The Meterpreter keylogging functionality was used to capture keyboard input from the compromised system. This demonstrated the ability of attackers to monitor user activity and collect sensitive information after gaining system access.



11. Remote Screenshot Capture

The Meterpreter screenshot feature was used to remotely capture the victim machine’s desktop. This capability demonstrates how attackers may visually monitor user activities and collect sensitive information from compromised systems.



Key Findings

• Multiple outdated and vulnerable services were identified during scanning and enumeration.
• Reverse shell access was successfully established using Meterpreter payloads.
• The target system lacked sufficient endpoint protection against malicious executable payloads.
• Post-exploitation activities demonstrated the risks associated with weak system hardening and insufficient monitoring.

Recommendations

• Regularly update operating systems and vulnerable applications.
• Disable unnecessary network services and close unused ports.
• Deploy endpoint detection and response (EDR) solutions.
• Implement antivirus and application whitelisting policies.
• Conduct periodic vulnerability assessments and penetration tests.
• Educate users regarding phishing and malicious executable files.

Conclusion

The penetration testing exercise successfully demonstrated the complete lifecycle of a cyberattack within a controlled environment. The project highlighted the importance of proactive security measures such as patch management, endpoint protection, network monitoring, and secure configurations to reduce exposure to cyber threats.